

NHẬN DẠNG CHỮ SỐ VIẾT TAY MNIST

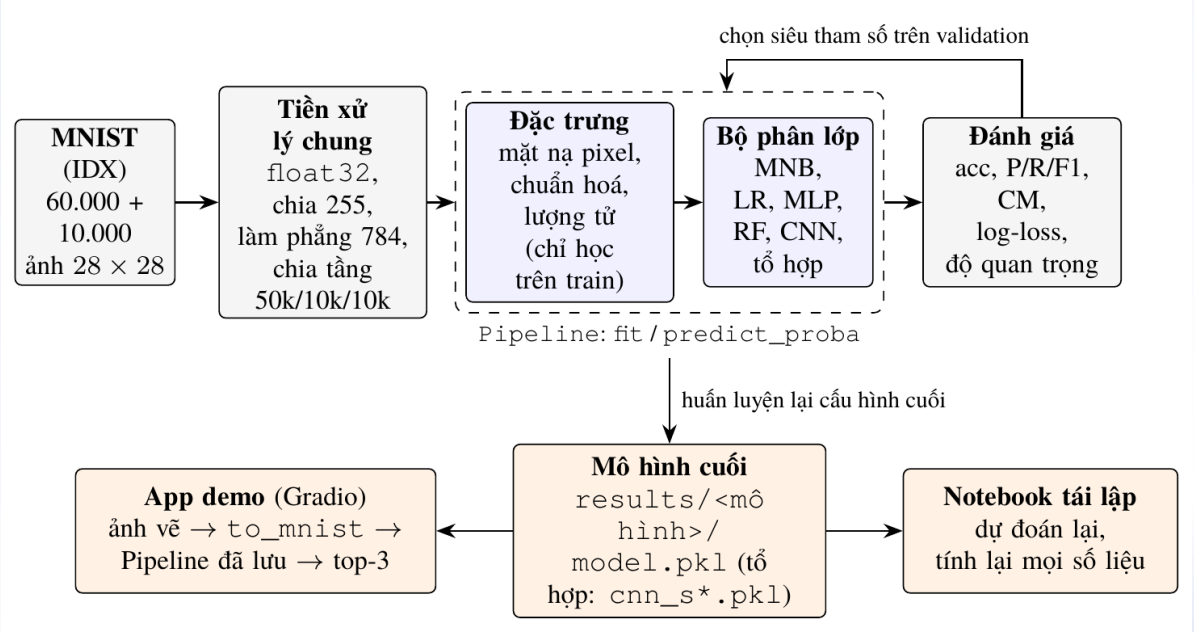
So sánh Naive Bayes, hồi quy softmax, MLP, Ensemble và CNN

TÌM HIỂU DỮ LIỆU

Dữ liệu và tiền xử lý chung

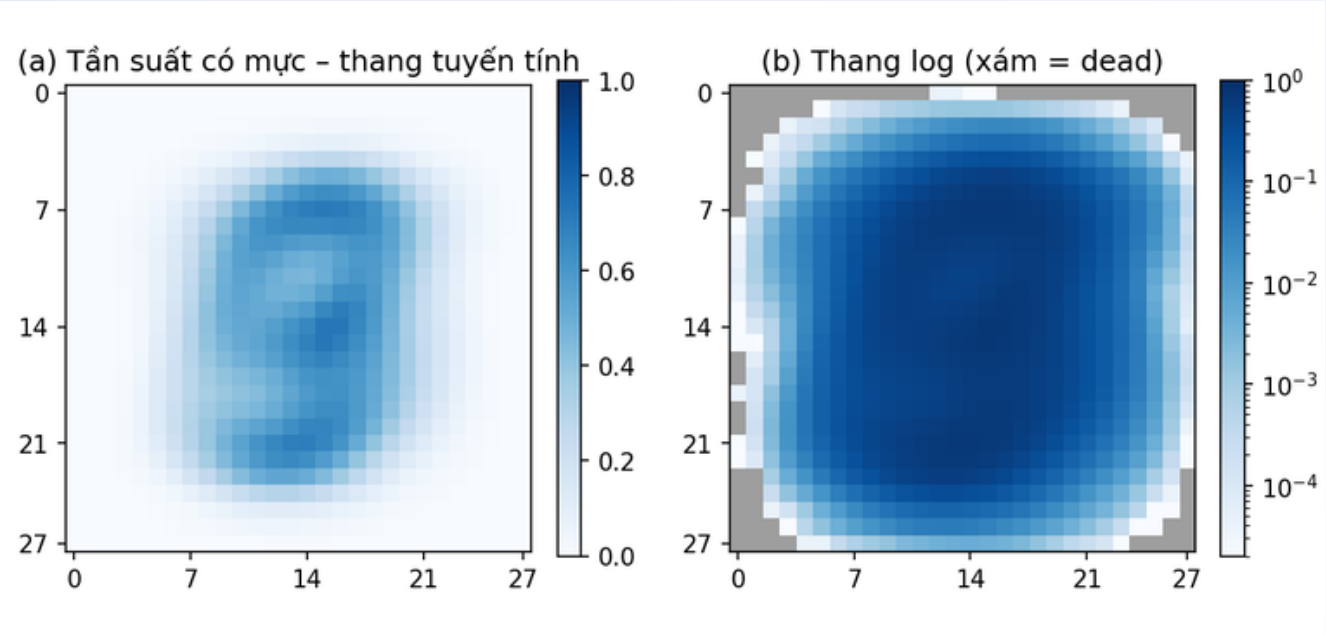
Chỉ các bước xử lý thô

- MNIST gốc định dạng IDX: 60.000 ảnh train, 10.000 ảnh test, 28×28
- Ép kiểu float32 trước khi chia
- Làm phẳng 28×28 → 784
- Tách 10.000 ảnh validation từ train theo tầng (stratified, seed 42)
- Mọi biến đổi có tham số (mask, chuẩn hoá, lượng tử) chỉ học trên train
- EDA làm trên 50.000 ảnh train





EDA nền: pixel gần nhị phân, viền gần như trống



(a) pixel gần nhị phân, viền gần như trống

80,86%

giá trị pixel bằng 0; 69,2% pixel khác 0 có giá trị ≥ 128

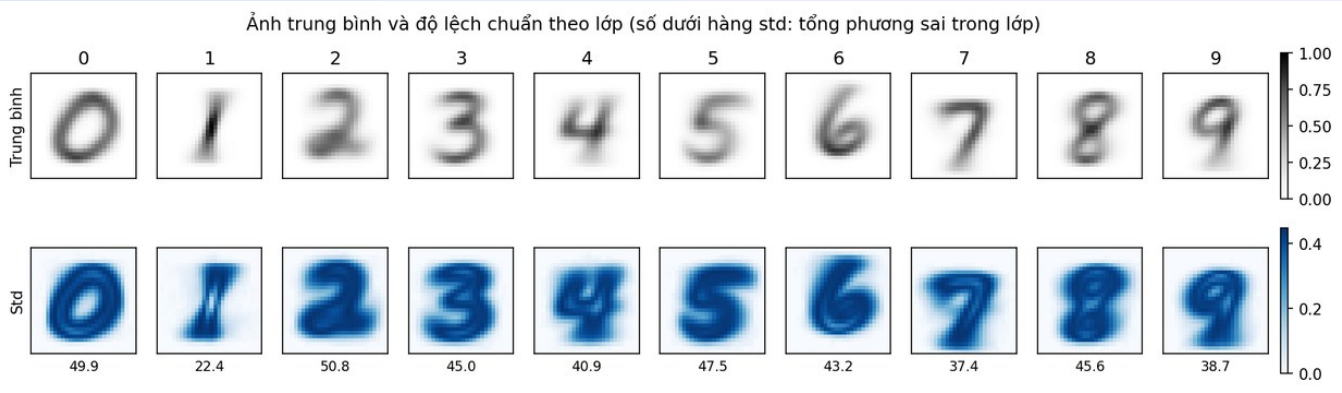
72 + 116

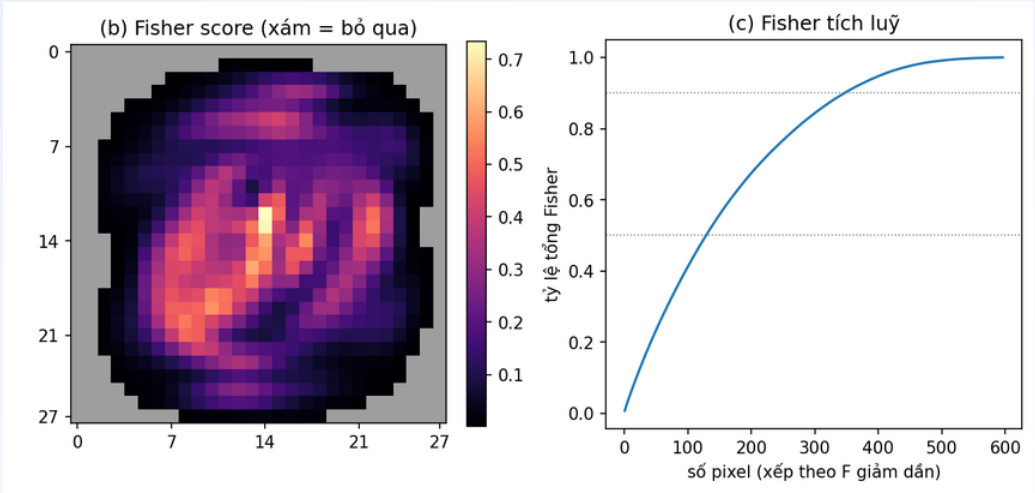
dead pixel (phương sai 0) và pixel có mực ở < 0,1% số ảnh, đều ở viền

99,76%

lượng mực nằm trong khung 24×24 ở giữa

EDA nền: tương quan, hình học lớp, Fisher





(b) Tương quan cục bộ

- Pixel kề nhau tương quan 0,708; ≈ 0 ở khoảng cách 5
- Biết lớp vẫn còn 0,64 - 0,86 → giả định độc lập của Naive Bayes bị vi phạm

(c) Lớp khác cả tâm lẫn phân tán

- Phương sai trong lớp 22,4 (lớp 1) → 50,8 (lớp 2): ranh giới không tuyến tính
- 5 cặp tâm gần: 4 - 9, 7 - 9, 5 - 8, 3 - 5, 3 - 8

(d) Thông tin phân biệt dồn ở giữa

- 129 pixel giữ một nửa tổng Fisher

$$F_j = \frac{\sum_c n_c \left(\mu_{cj} - \mu_j\right)^2}{\sum_c n_c \sigma_{cj}^2}$$

Phát hiện	Quyết định
(a) Pixel gần nhị phân	MNB trên cường độ [0, 1]; RF nhị phân hoá
(a) 188 pixel chết/hiếm	Mặt nạ pixel; Laplace $\alpha > 0$; chuẩn hoá vô hướng
(b) Tương quan cục bộ	L2; độ quan trọng theo khối 4×4; tích chập 3×3
(c) Ranh giới phi tuyến	So RF, MLP với LR (H2)
(c) 5 cặp tâm gần	Theo dõi riêng 5 cặp (H3)
(d) Fisher dồn ở giữa	So độ quan trọng với Fisher (H4)

H1 (b)
LR chính xác hơn và cho xác suất đáng tin hơn MNB

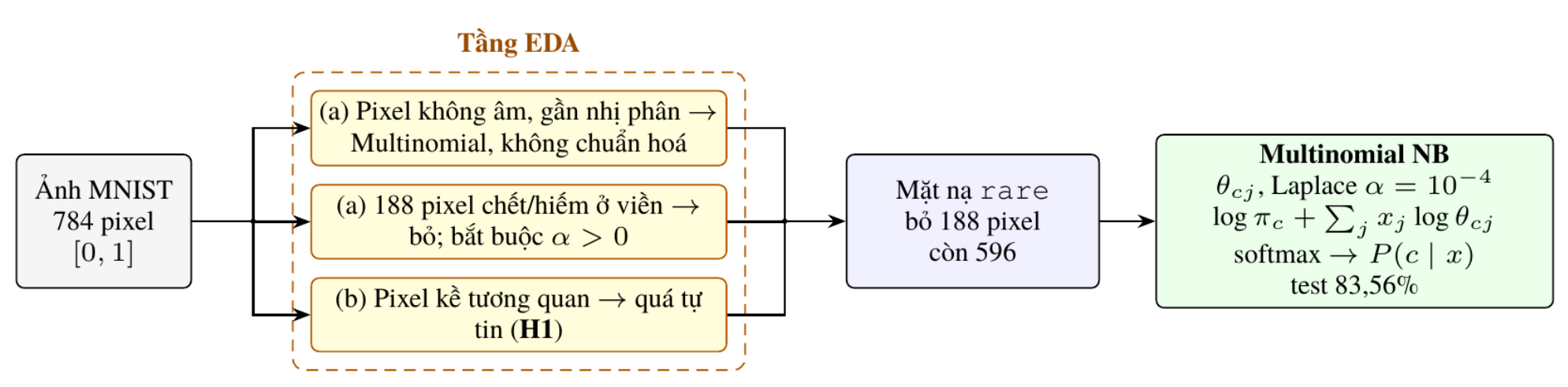
H2 (c)
RF và MLP vượt LR về accuracy và F1 macro

H3 (c)
Lỗi của mô hình tuyến tính dồn vào 5 cặp tâm gần (≥ 2 lần mức rải đều); mô hình phi tuyến giảm chính các lỗi đó

H4 (d)
Độ quan trọng theo khối tương quan Spearman ≥ 0,7 với Fisher

BỘ PHÂN LỚP

Multinomial Naive Bayes – Naive Bayes



Multinomial

- Pixel không âm, gần nhị phân, không có dạng chuẩn (phát hiện a)
- Coi cường độ pixel là "lượng mực" rơi vào vị trí j → giữ thang [0, 1], không chuẩn hoá
- Hàm quyết định tuyến tính theo x (trọng số log θ): cùng dạng với hồi quy softmax

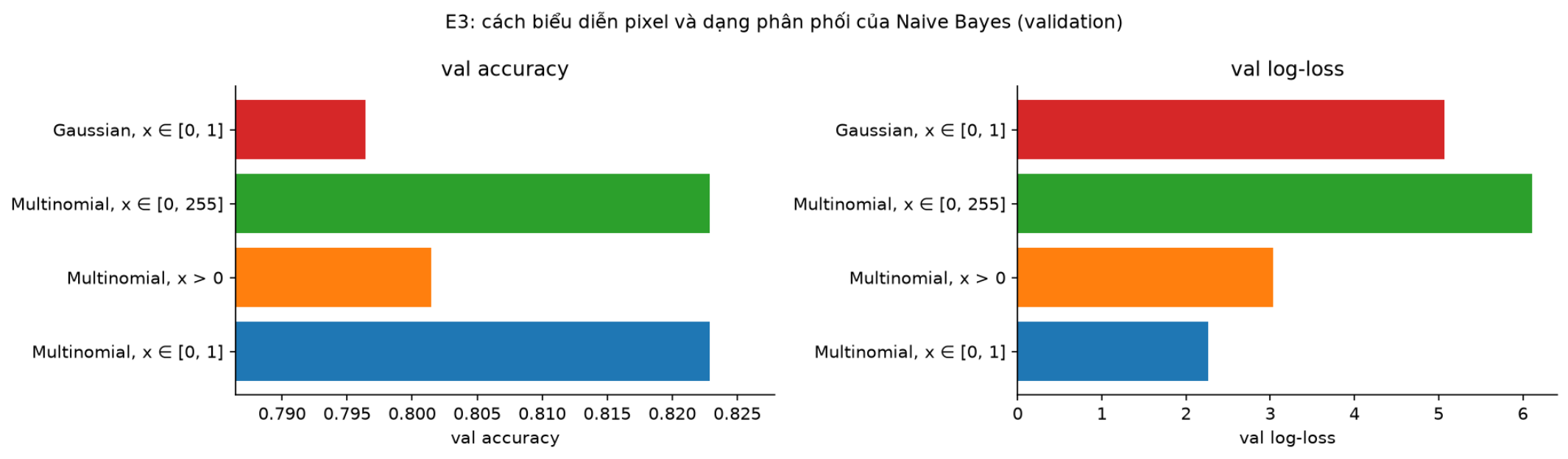
Nền tảng

$$\theta_{cj} = \frac{N_{cj} + \alpha}{\sum_{j'} N_{cj'} + \alpha d}$$

$$\log P(c|x) \propto \log \pi_c + \sum_j x_j \log \theta_{cj}$$

- α > 0 bắt buộc: pixel hiếm có N_{cj} = 0
- Tính trong miền log, xác suất qua log-sum-exp

Multinomial Naive Bayes: chọn tham số



E3: accuracy và log-loss trên val theo cách biểu diễn pixel, so với Gaussian NB

E1 · làm trơn α ∈ {10⁻⁴ … 10}

- Val 0,8205 → 0,8228, chênh 23 ảnh
- Chọn α = 10⁻⁴: giới hạn không nằm ở θ

E2 · bỏ pixel (từ a)

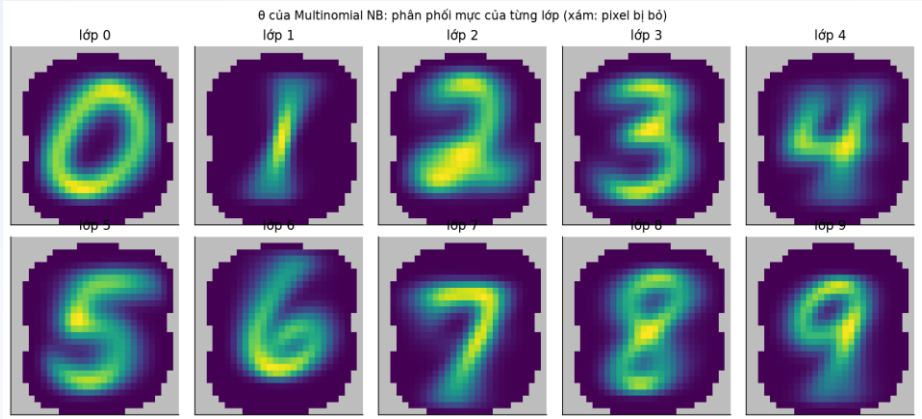
- Bỏ chết / hiếm / cắt 24×24: 0,8218–0,8229
- Cắt 20×20 tụt 0,8126 → chọn mặt nạ rare, còn 596 pixel

E3 · biểu diễn pixel

- [0, 1] hơn nhị phân 2,1 điểm; ×255 đẩy log-loss 2,26 → 6,11
- Gaussian NB 0,7964, 87,4% lỗi có p > 0,99 → giữ Multinomial

Multinomial Naive Bayes: scikit-learn

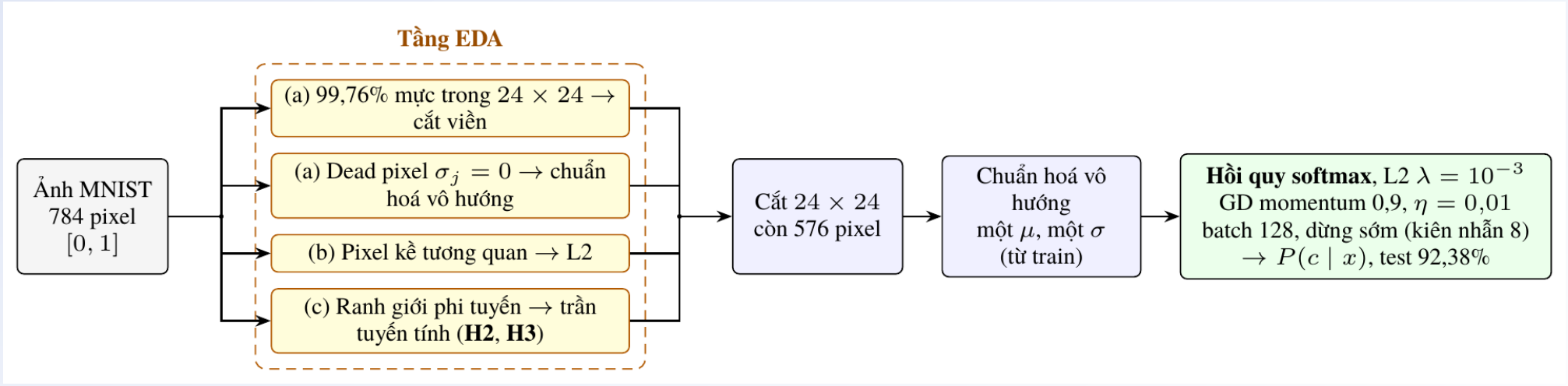
Tiêu chí (test)	Tự cài	MultinomialNB
Accuracy	0,8356	0,8356
F1 macro	0,8342	0,8342
Huấn luyện (giây)	0,5	2,9
Dự đoán 10.000 ảnh (giây)	0,05	0,16



Kết luận

- Khớp tuyệt đối; xác suất khớp tới 10^{-10}
- Giải thích trực tiếp nhất: θ_c chính là ảnh chữ số

Hồi quy softmax – Phân lớp tuyến tính



Hồi quy softmax

- Cần xác suất để kiểm H1 → softmax, không SVM
- Không giả định pixel độc lập, nhưng ranh giới tuyến tính: biết trước có trần (phát hiện c)

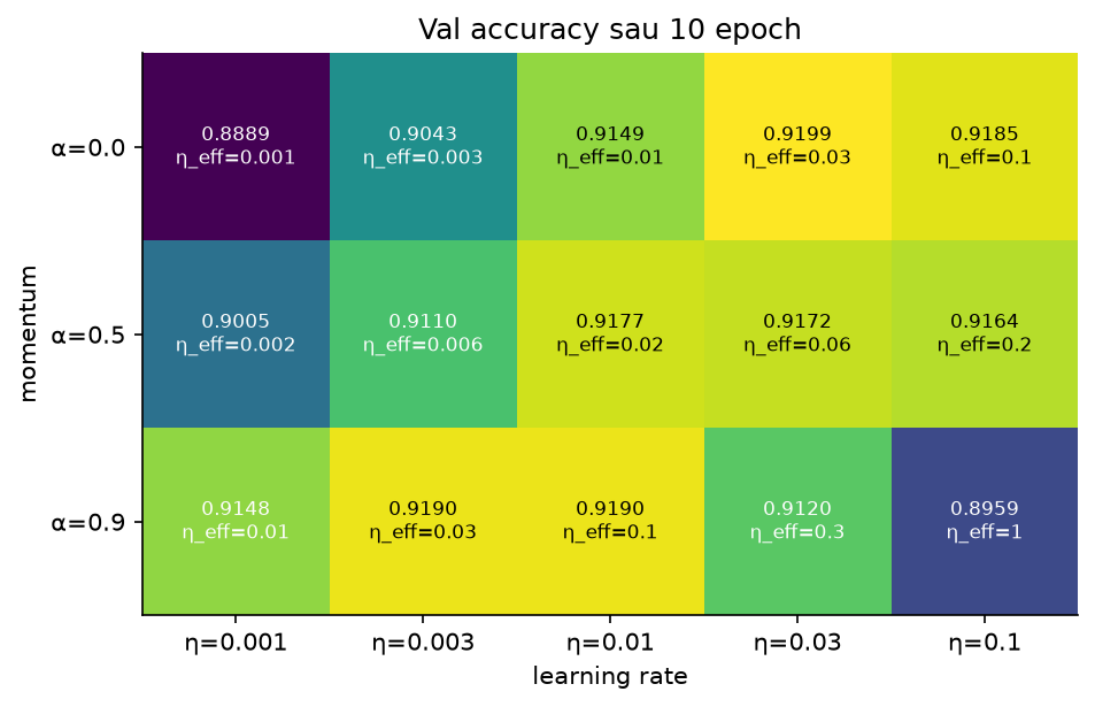
Nền tảng

$$J(W) = -\frac{1}{N}\sum_{i} \log p_{i,y_i} + \frac{\lambda}{2}\|W\|_F^2$$

$$\nabla J = \frac{1}{N}\tilde{X}^{\top}(P - Y) + \lambda M \odot W$$

- J lồi → khởi tạo W = 0; không phạt bias (mặt nạ M)
- Mini-batch có momentum, dừng sớm theo val loss; gradient so với sai phân trung tâm

Hồi quy softmax: chọn tham số



E2a · learning rate × momentum

- Đại lượng quyết định là bước hiệu dụng; 0,02 - 0,1 tốt nhất, bằng 1 thì sụt về 0,896
- Chọn μ = 0,9, η = 0,01 (val loss 0,2919)

$$\eta_{\text{eff}} = \frac{\eta}{1-\mu}$$

E1 · chuẩn hoá (từ a)

- Không chuẩn hoá 0,9155 · z-score từng pixel 0,9182 (|z| tới 1314)
- Vô hướng (một μ, một σ) 0,9210 → chọn

E2b - c · batch

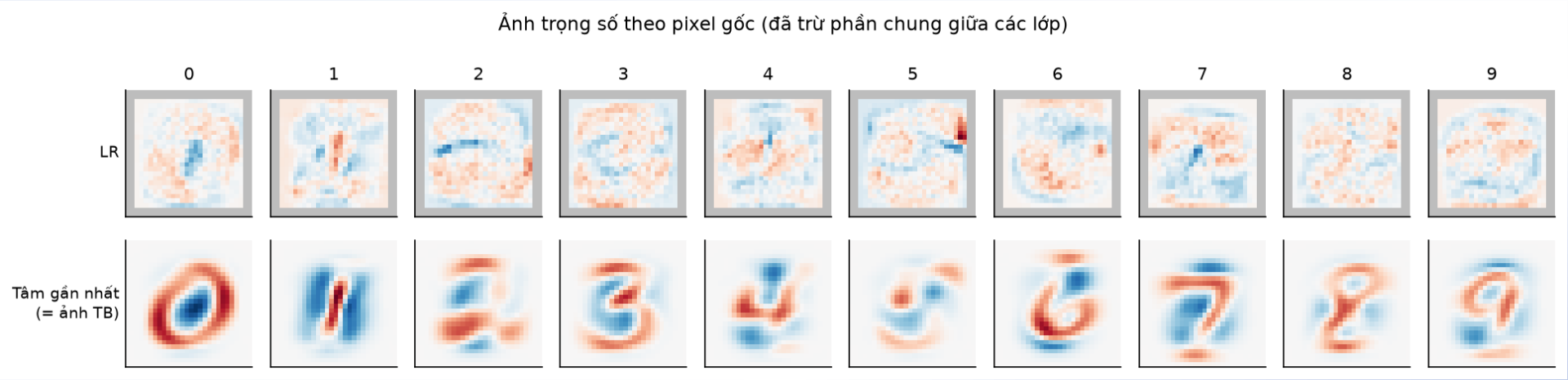
- Sau 10 epoch, batch 128 có J = 0,258; full-batch cần 100 epoch để tới 0,257 → chọn 128

E3 · L2 (từ b)

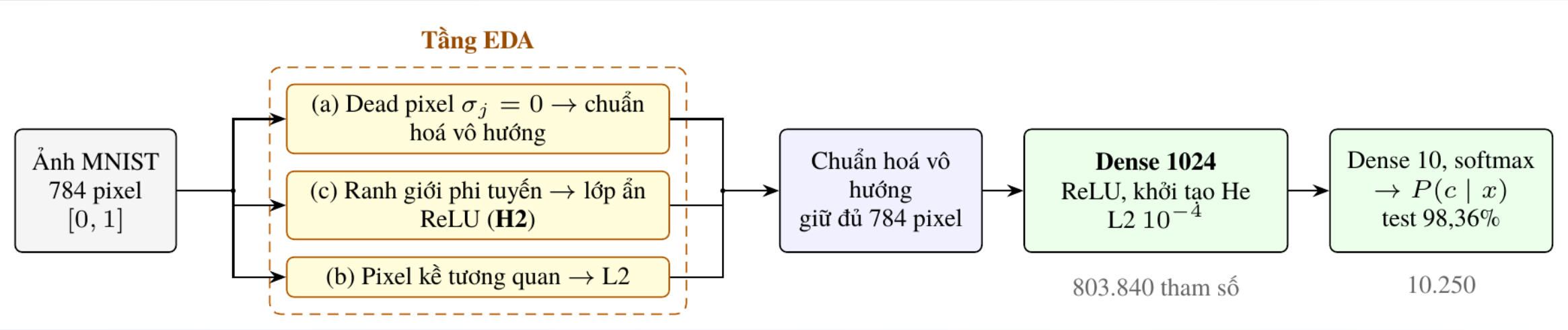
- $\lambda \le 10^{-3}$ giữ 0,920 - 0,922; 10^{-2} thiếu khớp
- Chọn 10^{-3}: val loss 0,2872, ‖W‖ 11,1 → 6,7

Hồi quy softmax: scikit-learn

Tiêu chí (test)	Tự cài	MLPClassifier, không lớp ẩn	LogisticRegression (lbfgs)
Accuracy	0,9238	0,9167	0,9238
F1 macro	0,9227	0,9155	0,9227
Huấn luyện (giây)	2,7	7,0	543
Dự đoán 10.000 ảnh (giây)	0,07	0,13	-



Multil Layer Perceptron



MLP

- Ranh giới phi tuyến (c): một lớp ẩn phi tuyến đã đủ biểu diễn
- Lớp 1 nối với từng pixel → nhiều tham số, dễ khớp nhiễu → cần điều chuẩn (b)
- Chuẩn hoá vô hướng như LR, giữ đủ 784 pixel

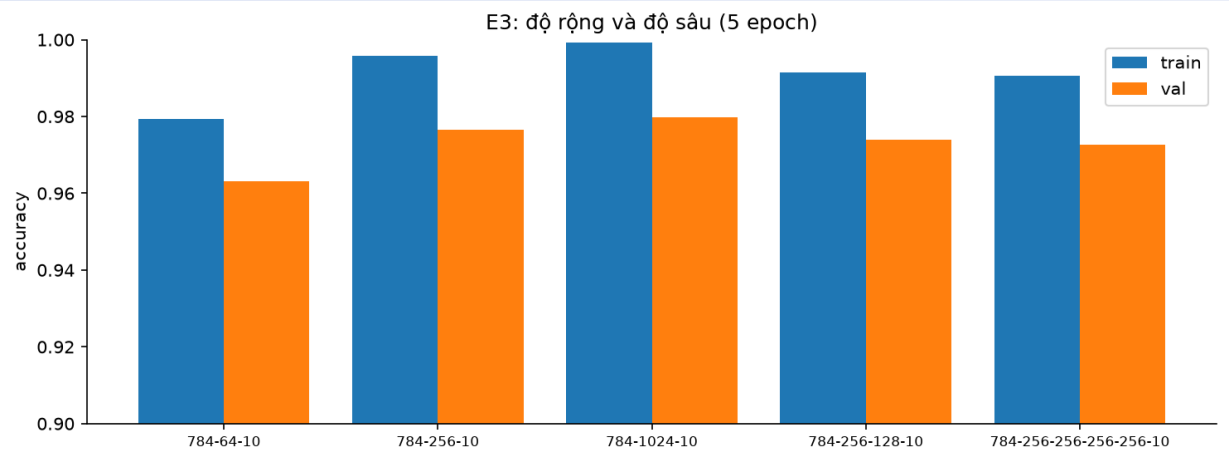
Nền tảng

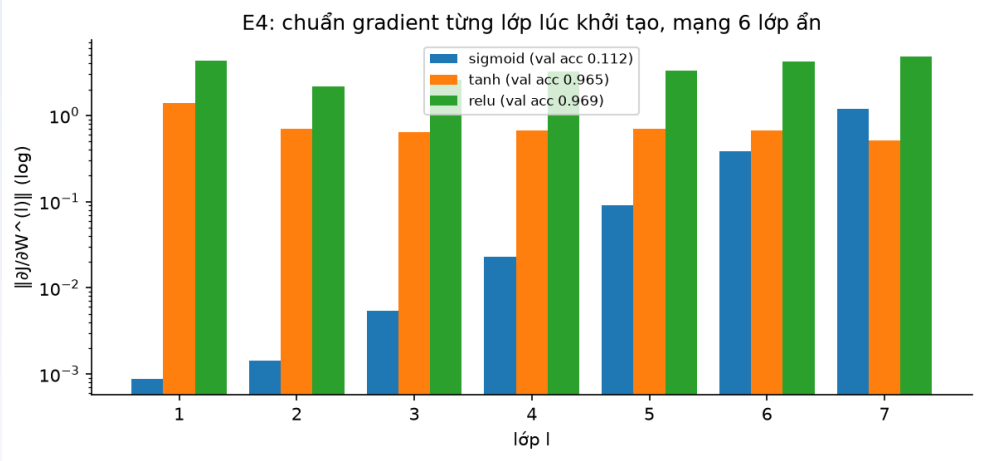
$$\delta^{(L)} = \frac{1}{B}(\hat{Y} - Y), \qquad \delta^{(l)} = \left(\delta^{(l+1)} W^{(l+1)}\right) \odot g'\left(Z^{(l)}\right)$$

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l)\top} A^{(l-1)} + \lambda W^{(l)}$$

- NumPy: thuận, ngược, dropout, SGD momentum
- Gradient so với autograd của PyTorch: sai số < 10^{-8}

MLP: chọn tham số





E3 · rộng hay sâu

- 784 – 1024 – 10 đạt 0,9798
- 4 lớp ẩn 256 chỉ 0,9726
- → một lớp ẩn 1024

E4 - E5 · kích hoạt, khởi tạo

- Sigmoid 6 lớp: gradient lớp 1 nhỏ hơn 1.355 lần, acc 0,112
- He giữ std 0,95 ở lớp 10, Xavier 0,03 → ReLU + He

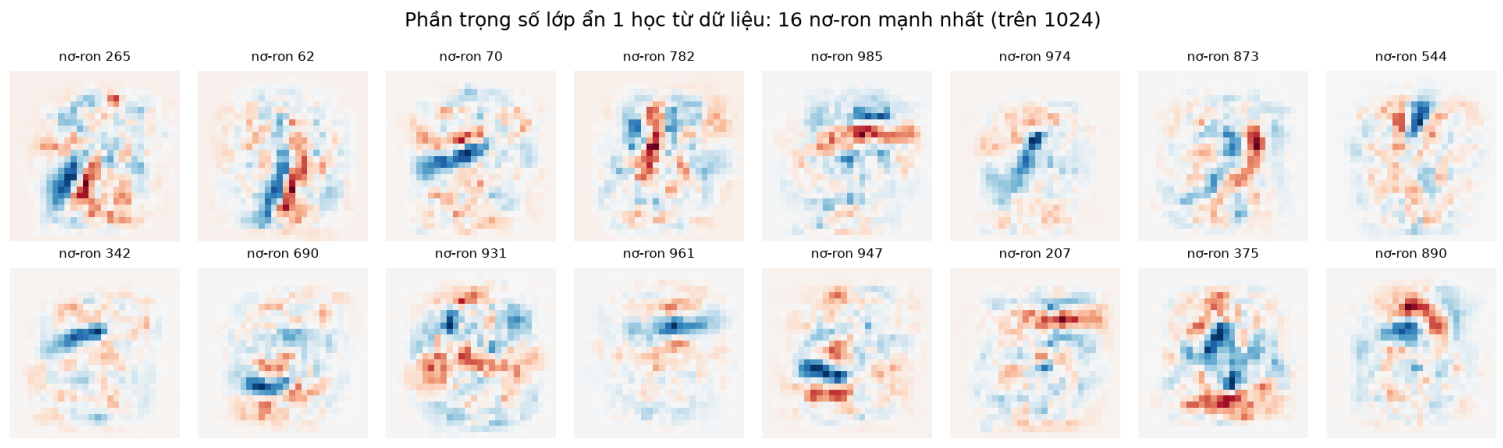
E6 · điều chuẩn (từ b)

- L2 10^{-4}: 0,9818
- Dropout 0,2: 0,9770
- Không điều chuẩn: 0,9769

E7 · batch

- Batch 256 dừng ở epoch 14: 0,9802
- Batch 128 học tới epoch 42: 0,9831 → chọn
- 814.090 tham số

MLP: scikit-learn

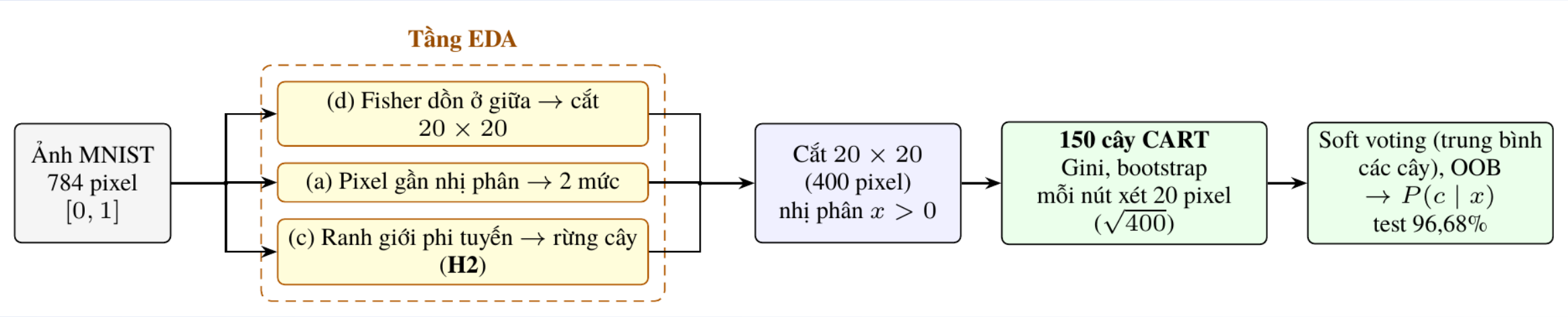


Tiêu chí (test)	Tự cài	MLPClassifier
Accuracy	0,9836	0,9823
F1 macro	0,9835	0,9822
Huấn luyện (giây)	248	574
Dự đoán 10.000 ảnh (giây)	0,14	0,17

Kết luận

- Chênh 0,13 điểm (khởi tạo, dãy trộn)
- Tự cài huấn luyện nhanh hơn hơn 2 lần

Random Forest – Ensemble



Random Forest

- Cây chia không gian thành nhiều vùng → bắt được ranh giới phi tuyến (c)
- Bagging + chọn ngẫu nhiên đặc trưng ở mỗi nút → cây ít giống nhau, trung bình giảm phương sai
- Chọn RF thay boosting: cây độc lập, song song, có OOB

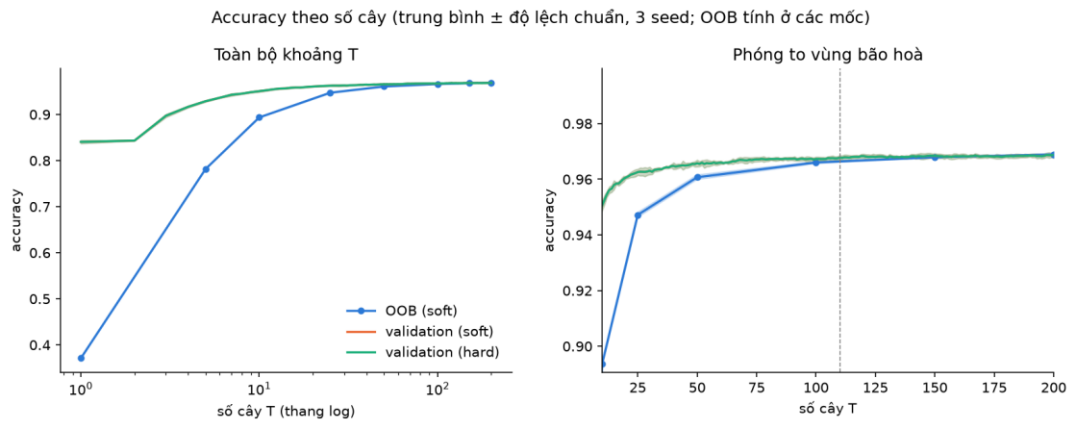
Nền tảng

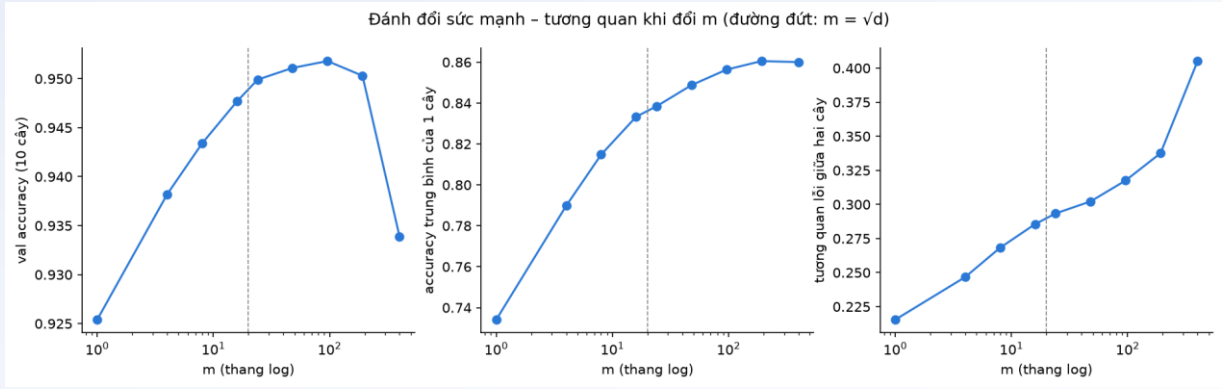
$$G = 1 - \sum_{c} p_c^2$$

$$P(c|x) = \frac{1}{T}\sum_{t=1}^{T} P_t(c|x)$$

- RandomForestClassifier của sklearn
- mask, lượng tử (a, d) và phân tích rừng, kiểm bằng tính vét cạn

Random Forest: chọn tham số





E1 · số mức lượng tử

- 2 - 256 mức: 0,9588 - 0,9608, trong dao động (std seed 0,0009)
- → 2 mức (có mực), nhanh hơn 1,35 lần

E2 · bỏ pixel

- Mọi mặt nạ trong dao động
- → cắt 20×20 còn 400 pixel, thời gian còn 65%

E3 · số cây

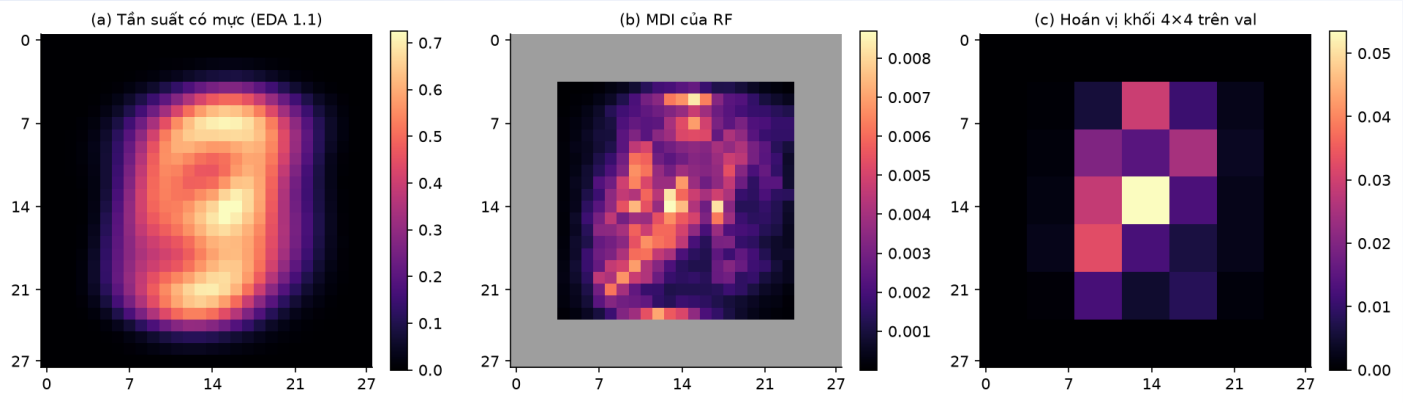
- Từ 110 cây, val trong 0,1 điểm của 200 cây → 150
- OOB thấp khi ít cây (≈ 37% số cây bỏ phiếu)

E4 · m pixel mỗi nút

- m = d (bagging): tương quan lỗi 0,41, kém đỉnh 1,8 điểm
- 50 cây: √d hơn m = 96 0,12 điểm, nhanh 2,4 lần

Random Forest

Tiêu chí (test)	Random Forest	HistGradientBoosting	AdaBoost (cây sâu 3)
Accuracy	0,9668	0,9781	0,9141
Huấn luyện (giây)	7,3 (14 luồng)	150	—



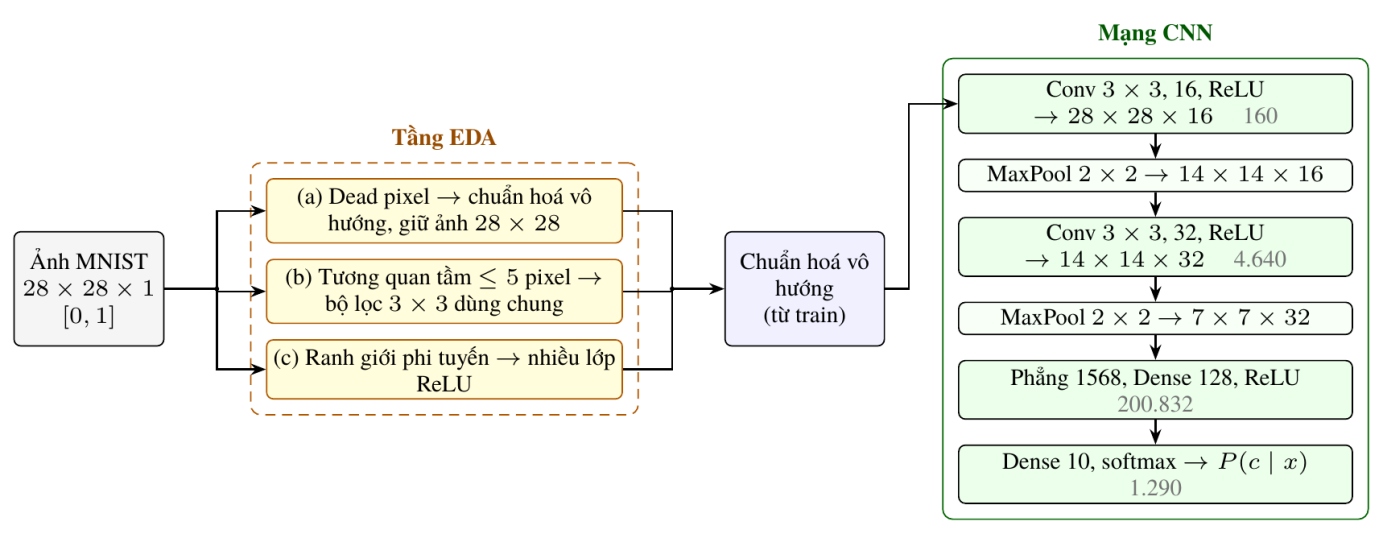
Kết luận

- Boosting hơn 1,1 điểm nhưng lâu gấp hơn 10 lần (học tuần tự)
- RF: F1 macro 0,9666, dự đoán 0,15 giây, mô hình ≈ 220 MB

MỞ RỘNG

CNN · CNN × 5 · demo

CNN: bộ lọc dùng chung cho tương quan cục bộ



Tiêu chí (test)	Tự cài (NumPy)	torch.nn
Accuracy	0,9882	0,9888
F1 macro	0,9881	0,9887
Huấn luyện (giây)	273	84
Dự đoán (giây)	2,88	0,56

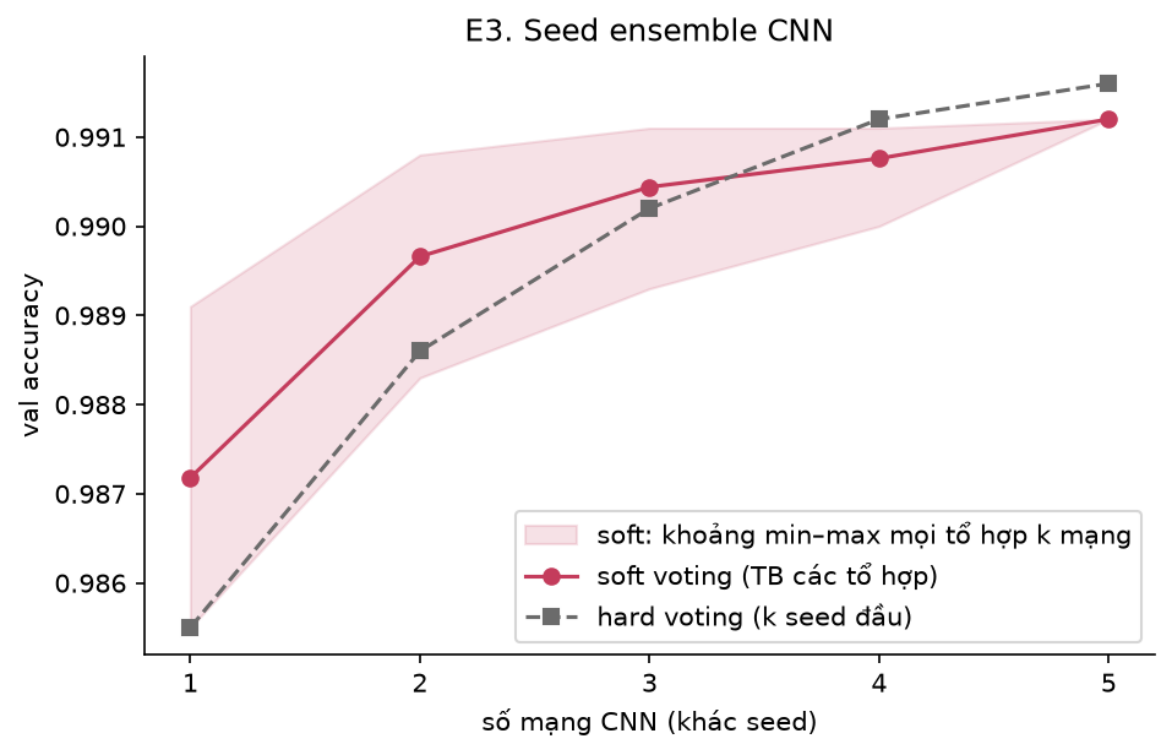
Vì sao thêm CNN?

- Tương quan chỉ trong tầm ≈ 5 pixel (b): bộ lọc 3×3 đủ bao, dùng chung mọi vị trí
- 206.922 tham số, ít hơn MLP gần 4 lần

Xây dựng và chọn tham số

- NumPy: im2col, lan truyền ngược, Adam; so với PyTorch
- Adam: $3{\cdot}10^{-4}$ 0,9788 · 10^{-3} 0,9837 · $3{\cdot}10^{-3}$ 0,9809
- Bộ lọc (16, 32): 0,9837 trong 101 giây
- Dừng sớm: tốt nhất epoch 5, dừng epoch 8

CNN × 5: trung bình 5 mạng khác seed



$$\overline{P}(c|x) = \frac{1}{5}\sum_{k=1}^{5} P_k(c|x)$$

118 → 85
ảnh sai trên test (−28%)

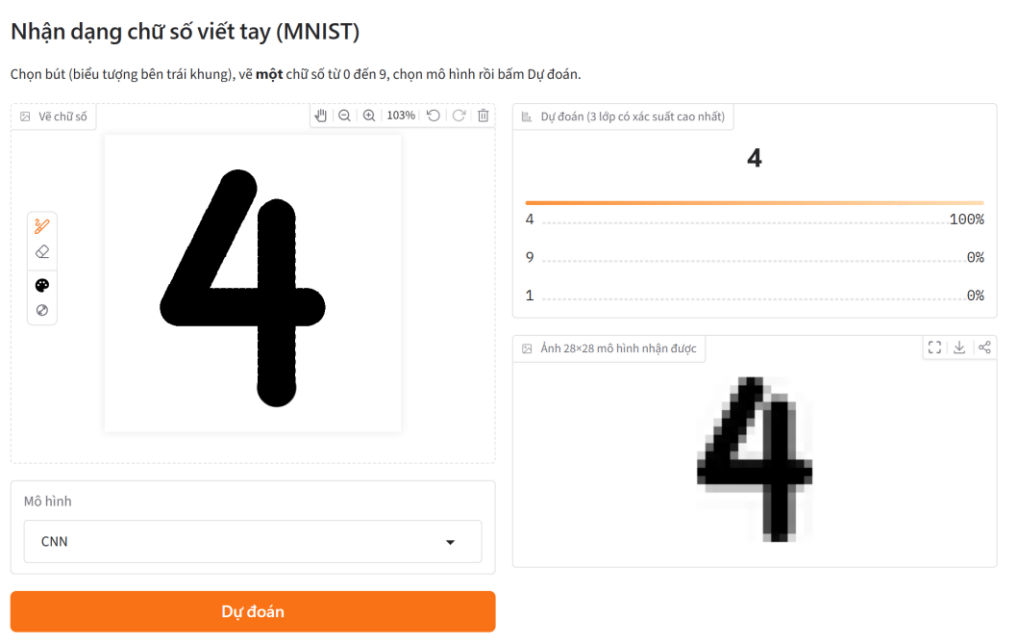
99,15%
accuracy test của CNN × 5

Khoảng 40 cách gộp thử trên val

- 5 seed CNN: val 0,9855 - 0,9891; seed 0 kém nhất
- Gộp mô hình cổ điển không có lợi: MNB sai khác CNN (cùng sai ≈ 5%) nhưng quá tự tin
- Thêm RF không sửa thêm ảnh nào, log-loss 0,025 → 0,041

→ dùng 5 CNN một mình

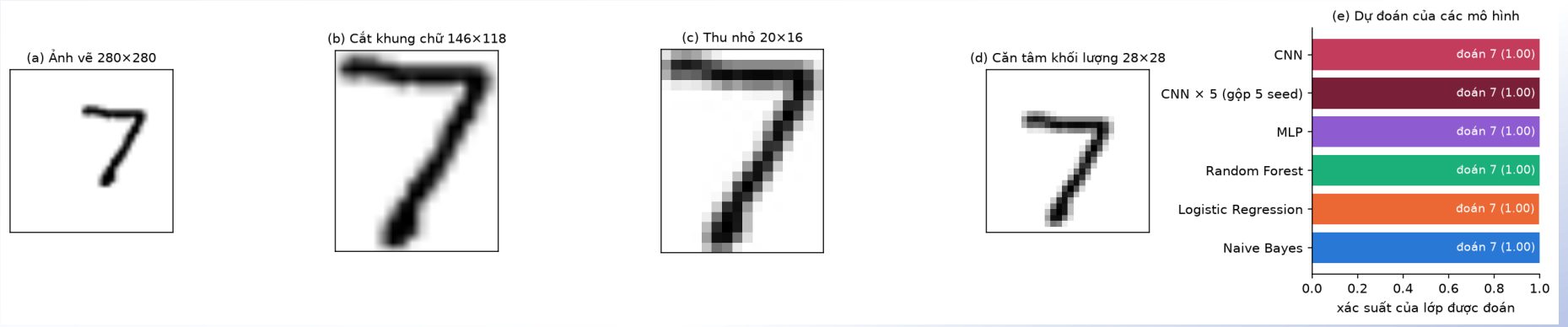
Demo: đưa ảnh vẽ về đúng quy ước MNIST



to_mnist (demo/drawing.py)

1. Đặt lên nền trắng, đảo màu: mực m ∈ [0, 1]
2. Cắt khung chứa nét, thu nhỏ giữ tỉ lệ cho cạnh dài 20
3. Căn tâm khối lượng vào giữa khung 28×28
4. Bỏ quầng mờ (m < 0,05), đưa vào chính Pipeline đã lưu

300 ảnh test vẽ to ×7, lệch ±40 pixel: CNN giữ 99,3% dự đoán; thu nhỏ thẳng cả khung chỉ còn 54,0%.



TỔNG KẾT VÀ SO SÁNH

Kết quả của các mô hình cuối

Mô hình	Acc train	Acc val	Acc test	Log-loss	Huấn luyện (s)	Dự đoán (s)	Luồng
Tâm gần nhất (mốc)	0,8080	0,8048	0,8198	—	0,2	0,02	1
Multinomial NB	0,8247	0,8229	0,8356	2,007	0,5	0,05	1
Hồi quy softmax	0,9281	0,9197	0,9238	0,276	2,7	0,07	8
Random Forest	1,0000	0,9694	0,9668	0,218	7,3	0,15	14
MLP	1,0000	0,9831	0,9836	0,054	247,7	0,14	8
CNN	0,9963	0,9855	0,9882	0,039	273,1	2,88	8
CNN × 5	-	-	**0,9915**	0,025	-	-	8

- Thứ tự giống nhau trên val và test; P, R, F1 macro chênh nhau ≤ 1,2 điểm vì 10 lớp gần cân bằng
- Các phần H1 - H4 dùng số trên validation; test chỉ dùng một lần cho mô hình cuối

H1: Multinomial NB quá tự tin

Trên val	MNB	LR
Accuracy	0,823	0,920
Độ tự tin trung bình	0,978	0,905
Log-loss	2,259	0,291
Ảnh sai với p > 0,99 (trên mọi ảnh)	10,35%	0,19%

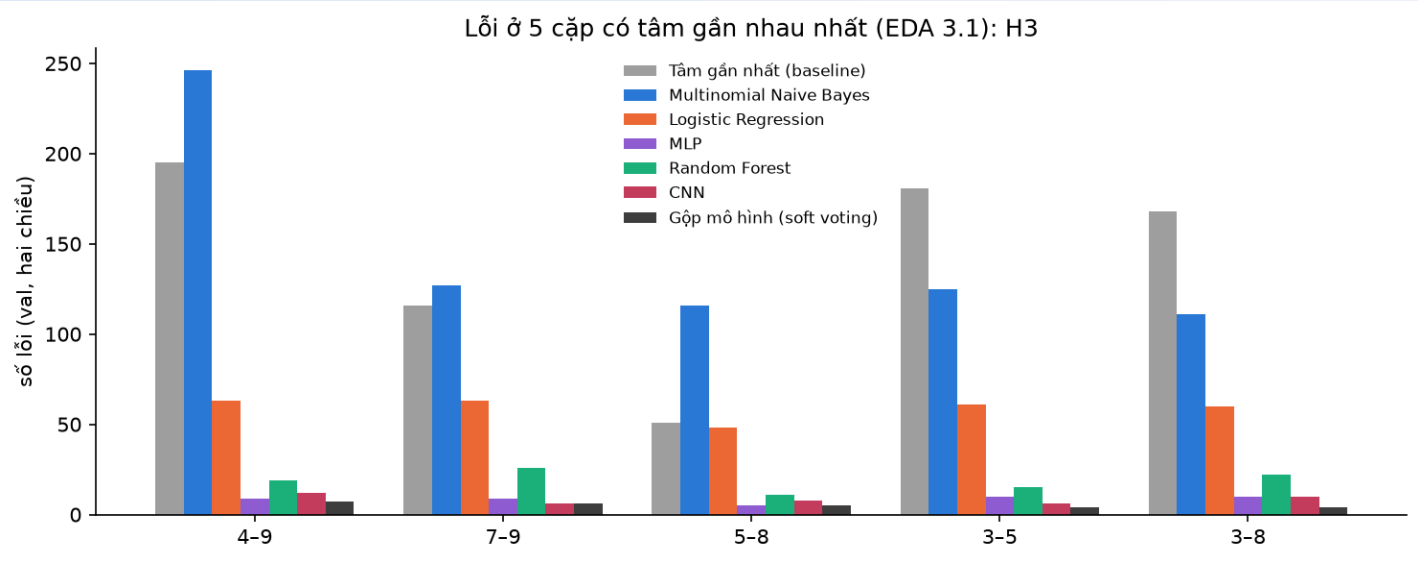
58,4%
lỗi của MNB có xác suất > 0,99 (LR: 2,4%)

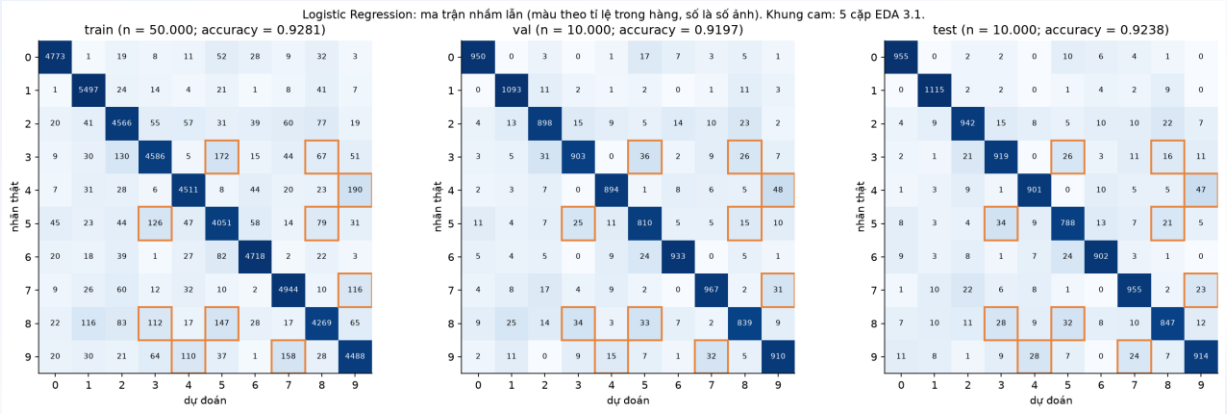
0,978
tự tin trung bình khi chỉ đúng 82,3%

Vì sao (phát hiện b)

- MNB và LR cùng dạng hàm tuyến tính → khác biệt chỉ còn ở cách tìm trọng số
- MNB cộng bằng chứng từ các pixel tương quan như thể độc lập: cùng một bằng chứng bị đếm nhiều lần
- Tăng α không sửa được vì lỗi nằm ở giả định → H1 được ủng hộ

H2, H3: lỗi tuyến tính dồn vào 5 cặp tâm gần





11%
nếu lỗi rải đều

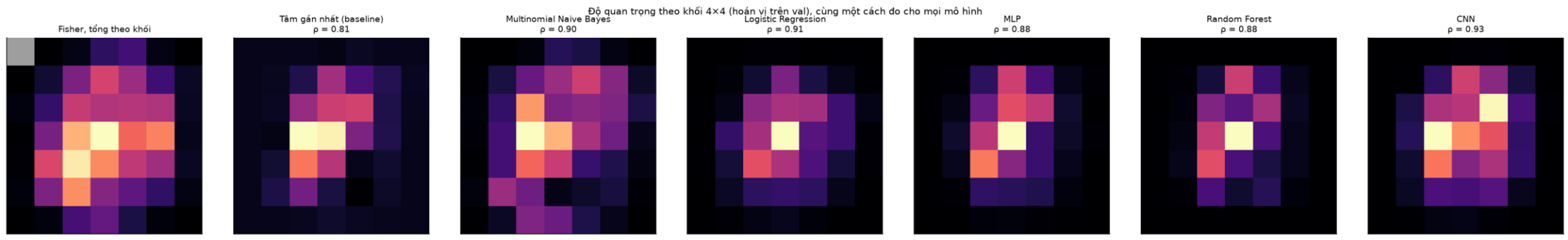
41% · 37%
thực tế ở MNB · LR

Kết quả

- H2: RF (0,9694), MLP (0,9831) vượt LR (0,9197) cả accuracy lẫn F1 macro
- H3: cả 5 cặp nằm trong top 5 cặp nhầm của MNB và LR, đúng 5 cặp EDA chỉ ra
- So với LR: RF giảm 68% lỗi ở 5 cặp (tổng 62%); MLP giảm 85% (tổng 79%)

  → mô hình phi tuyến giảm chọn lọc đúng chỗ đường thẳng không tách được

H4: mọi mô hình nhìn vào cùng vùng giữa ảnh



0,90
Spearman với Fisher · MNB

0,91
Spearman với Fisher · LR

0,88
Spearman với Fisher · MLP

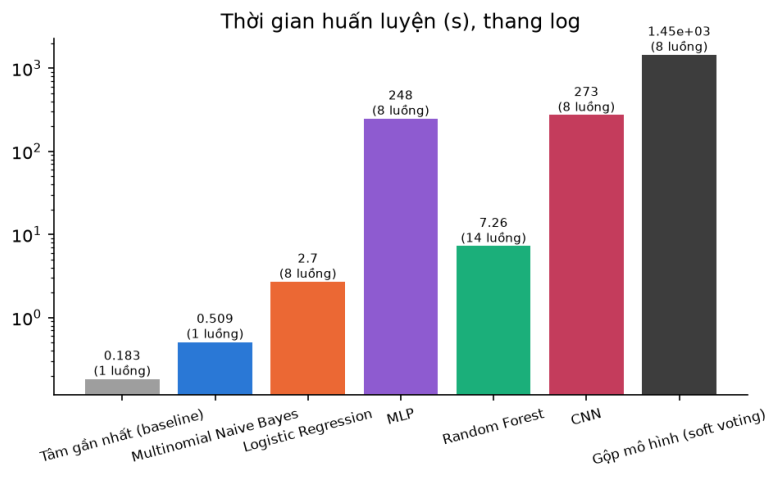
0,88
Spearman với Fisher · RF

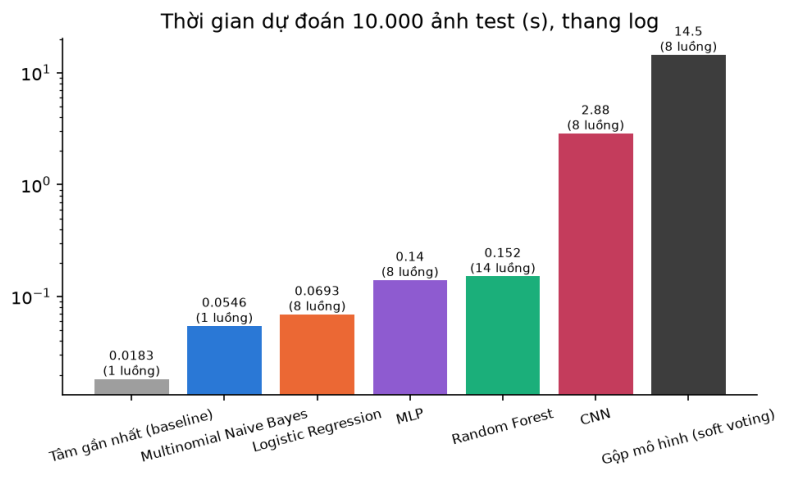
0,93
Spearman với Fisher · CNN

- Đều vượt ngưỡng 0,7 chốt trước → H4 được ủng hộ: năm họ mô hình khác nhau nhưng cùng dựa vào vùng có Fisher cao (d)
- Khối quan trọng nhất ở giữa ảnh; hoán vị nó làm accuracy val giảm 3,6 (MNB) đến 8,5 điểm (LR)

So sánh theo 4 chỉ số

	Multinomial NB	Hồi quy softmax	MLP	Random Forest
Độ chính xác	Thấp (83,6%), quá tự tin	Trung bình (92,4%), có trần	Cao nhất (98,4%)	Cao (96,7%), thiếu tự tin
Huấn luyện	0,5 s, một lần đếm	2,7 s, hàm lồi	248 s, nhiều tham số	7,3 s, song song
Dự đoán	0,05 s	0,07 s	0,14 s	0,15 s, ≈ 220 MB
Giải thích	Trực tiếp: θ_c	Ảnh trọng số lốm đốm	Gián tiếp: lớp 1	MDI, 150 cây





Accuracy không đủ

- RF và MLP accuracy gần nhau nhưng lệch hai hướng
- MLP hơi quá tự tin, RF tự tin 0,845 khi đúng 0,969

Kiểm chứng phát hiện ban đầu

Multinomial NB	Hồi quy softmax	RF so với MLP
Giả định Đổi α hay mặt nạ không giúp gì; bằng chứng tương quan bị đếm như độc lập *phát hiện (b)*	**Dạng hàm** Lỗi dồn vào các cặp lớp mà ranh giới tuyến tính không tách được *phát hiện (c)*	**Cách dùng đặc trưng** Mỗi phép chia nhìn một pixel nhị phân; MLP tổ hợp nhiều pixel ở lớp đầu *phát hiện (a)*

Kết luận

- Bốn bộ phân lớp cài đúng, khớp thư viện trong 0,13 điểm; cả bốn giả thuyết từ EDA đều được ủng hộ
- Mỗi bước tăng accuracy, từ MNB tới CNN × 5, đều trả bằng thời gian và khả năng giải thích
- Hiểu dữ liệu trước khi chọn mô hình cho biết không chỉ mô hình nào thắng, mà còn vì sao

CẢM ƠN THẦY VÀ CÁC BẠN